

Quote: Where traditional learning ends, deep learning begins, turning raw data into intelligence, and intelligence into discovery.

Prerequisites:

Linear Algebra

Probability & Statistics

Python Programming

ML Basics (Regression, Classification, Overfitting)

What is deep learning?

Imagine your brain has lots of tiny switches (neurons). When you see a cat, your brain looks at: the shape of the ears, the whiskers, the fur, the sound 'meow'. After seeing many cats, your brain gets really good saying; that's a cat.

Deep Learning works the same way. It builds layers of tiny switches (artificial neurons) inside a computer. Each layer learns something;

first layer; this looks like an edge.

next layer; this looks like an ear.

final layer; oh! its a cat.

Imagine you have a robot teacher. You give it your maths homework;

at first, it guesses randomly(lots of mistakes).

every time its wrong, you say NO, that's wrong, the answer should be 10 or 8.

the robot teacher fixes its thinking switches a little.

after many tries, it learns to get most answers correct.

That's deep learning, learning from mistakes until it gets really good.

ML VS DL:

Machine learning is like teaching a kid with rules and hints.

example; you want a kid to recognize fruits.

- * You say; apples are usually red, round, and sweet. bananas are yellow and long.

- * The kid uses those given rules to guess. So in ML, we give the computer data + features(hints), and it learns patterns.

In short; ML = Learning with guidance (features are hand-made by us).

Deep learning is like giving the kid a lot of fruits without telling the rules.

- * You just say! here's 1,000 apples and 1,000 bananas figure it out yourself.

- * The kid's brain notices; first layer => oh, red and round often go together.

next layer => oh, yellow and long often go together.

* Eventually, the kid can say, That's an apple! without you ever telling the rules.

In short, DL = Learning the rules by itself (no hand-made features).

Why DL is popular now?

1) Data Explosion:

* In the past, we didn't have enough data for deep networks to be useful.

* Now we have a massive amount of data: images (Instagram, YouTube), text (social media, books, articles), and speech (calls, podcasts).

* Deep learning thrives on big data => the more it gets, the smarter it becomes.

2) Hardware Power (GPUs & TPUs):

* Training deep networks needs a lot of calculations (matrix multiplications).

- * Earlier, computers were too slow.
- * Today, GPU & TPUs can train huge models in hours, not years.
- * example: without GPUs, training something like GPT or image recognition models would take decades.

3) Automatic Feature Learning:

- * In machine learning, we had to manually choose features (like the length of an email to detect spam).
- * Deep learning learns features automatically from raw pixels, raw text, & raw sound.
- * This makes it powerful for unstructured data (images, videos, audios, text).

4) Better Performance:

- * Deep learning models consistently beat traditional ML models in accuracy for tasks like; image recognition, speech recognition, & natural language processing(NLP).

* example: Google Translate became much better after switching from ML to deep learning.

5) Versatility Across Domains:

* Works on almost every type of data: text, image, audio, timeseries.

* Same deep network idea powers self-driving cars, ChatGPT, face recognition, Netflix recommendations, and medical image analysis.

6) Research Breakthroughs:

* Innovations like CNNs, RNNs, LSTMs, Transformers, & GANs made deep learning super flexible.

* Each breakthrough solved a major limitation and opened new applications.

What will we practice?

Phase 1 - Foundations

Phase 2 - Core Mechanics

Phase 3 - Training Challenges

Phase 4 - Artificial Neural Networks (ANNs)

Perceptron:

The perceptron is the simplest type of artificial neuron, introduced by Frank Rosenblatt in 1958. It is the foundation of neural networks and deep learning.

It mimics how a biological neuron works, taking inputs, processing them, and giving an output.

table =>

Age	Cholestrol	BP	Heart Disease
28	150	110	1
36	120	90	0
42	180	160	1

Step 1:

formula: $y = mx + b \Rightarrow h_0(x) = 0_0 + 0_1x_1$

$$h_0(x) = 0_0 + 0_1x_1 + 0_2x_2 + 0_3x_3$$

Step 2:

Sigmoid Activation Function: $= 1/(1+e^{-z}) \Rightarrow z = (w_0 + w_1x_1 + w_2x_2 + w_3x_3)$

* Here we squash line by using sigmoid activation function.

Step 3:

Log Loss Function:

* Or we can use binary cross-entropy.

* Here we find loss, image we set some values on w_1, w_2 or w_3 , and they give some loss then to reduce that loss we may use gradient descent.

Step 4:

Optimizer: Here we use one curve and that is gradient descent.

* Let suppose we have 1 input and 1 output, as age and heart disease, in graph we have to find different values of w_1 , also find log loss as it keep coming, we keep reducing it. Then find the lowest point among

all of them; that point is known as the global minimum.

Now in deep learning, we'll do all above steps by using perceptron.

$$\Rightarrow x_1 \quad 0 \quad - w_1$$

$$\Rightarrow x_2 \quad 0 \quad - w_2 \quad \Rightarrow 3 \text{ input layers} \quad 1 \text{ output layer} \Rightarrow 0 \text{ - bias}$$

$$\Rightarrow x_3 \quad 0 \quad - w_3$$

Now what will we do, we simply connect all these layers together. And then we apply weights, like w_1 , w_2 , w_3 . okay, so in output layer we use bias.

The formula created: $(x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + \text{bias})$

Is this formula look familiar? YES: $(w_0x_1 + w_1x_2 + w_2x_3 + w_0)$

* This is a base diagram of perceptron, and this is a forward propagation.

The secret: What if we add one hidden layer between input layer and output layer? You know what that layer called, that's a neural network layer, and that layer is called **Artificial Neural Network** means **ANNs**.

Limitation Of Perceptrons:

- * When perceptron developed in 1958, it used step activation function, which was always used to give binary classification.

- * It just represents linear regression boundaries

Artificial Neural Network (ANNs):

It was made with the help of perceptron.

O

O

O

O

O

O

input	hidden	output
layer	layer	layer

Forward Propagation:

It is a way to calculate day in ANNs.

Data:

x1		x2		x3		y
20		1.0		30		1

2.0 => O

O (0.1 - Bias 1)

1.0 => O

O (0.05 - Bias 3)

O (-0.2 - Bias 2)

3.0 => O

input neurons:

formula: $hn1 = (x1 w1) + (x2w2) + (x3w3) + b$

$$hn1 = (2.0)(0.5) + (1.0)(0.2) + (3.0)(0.1) + 0.1$$

$$hn1 = 1.6$$

For binary result we apply sigmoid formula:

$$\sigma = 1/1+e^{-z}$$

$$\sigma = (1.6)$$

$$\sigma = 0.8320$$

$$hn2 = (2.0)(-0.3) + (1.0)(0.8) + (3.0)(0.5) + (-0.2)$$

$$hn2 = 0.8176$$

$$\begin{aligned} \text{output neuron:} &= [0.8320(1.2)] + [0.8176(-0.7)] \\ &+ 0.05 \\ &= 0.4761 \end{aligned}$$

Now, as we apply the sigmoid activation function, we also apply that formula;

$$\sigma(0.4761) \Rightarrow 0.6168$$

- * if the value is greater than 0.5, then it will be 1.
- * otherwise it will be 0.
- * So, finally our $y_{pred} = 1$

And this is what we call forward propagation, but this result can be better. For the betterment of the result, we can update the weights by using backward propagation.

For practicing deep learning, there are just 3 important things that we'll use repeatedly.

- 1) Activation Functions => (Forward Propagation)
- 2) Loss Function] (Backward
- 3) Optimizer(Gradient Descent, Relu)] Propagation)

Backward Propagation:

All right, so the predicted value is $\hat{y} = 0.6168$ and the original value is 1. There is a gap between values. Here, we'll calculate the loss using the log loss function.

formula: $\log \text{ loss} = [y * \log (y^\wedge) + (1-y) * \log (1 - y^\wedge)]$
 $= 0.494$

For minimization of loss, we update the weight;

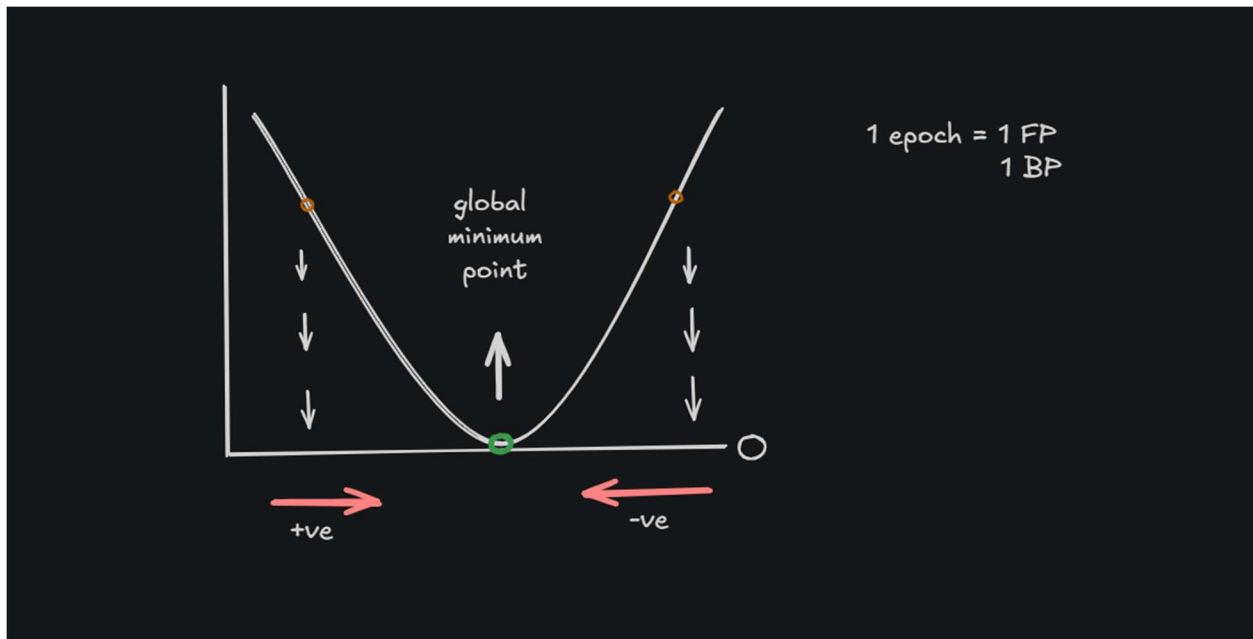
formula: $W \text{ new} = W \text{ old} - n * dL / dW \text{ old}$

expend => $(dL / dW \text{ old} = dL/dO3 * dO3/dW7 \text{ old})$

if we are update w1; $W1 \text{ new} = W1 \text{ old} - n (dL / dW1 \text{ old} = dL/dO3 * dO3/dW1 \text{ old})$

* This phase is called the chain rule of derivative.

* We can also use mean square error, but only with a regression problem; with classification, we only use the log loss function.



Vanishing Gradient Problem:

The vanishing gradient problem happens during backward propagation when gradients (the small updates we calculate to adjust weights) become so tiny that the earlier layers of a deep neural network stop learning.



* In the place of Sigmoid activation function, we'll use another function to get better value.

Types of Activation Function:

1) Linear Activation Function:

It is a simplest type of activation function,

$$f(x) = x$$

So, the output is the same as the input. The neuron doesn't transform the data it just passes it forward as it is. In regression tasks (prediction continuous values like salary, house price, temperature, etc).

Limitation:

* If you use linear activation in all layers, the whole network becomes just a linear model, no matter how many layers you add. That means it cannot capture complex pattern in data, that's why hidden layer use non-linear activations(ReLU, tanh, sigmoid), but the output layer for regression can be linear.

2) Sigmoid Activation Function:

It is a S-shaped curve function that squashes any real number into a range between 0 and 1.

$$f(x) = 1 / (1 + e^{-z})$$

Use in output layer, binary classification problems (yes/no, disease/no disease, spam/not spam).

Limitation:

* Vanishing gradient problem, for large or very small inputs, the gradient becomes almost 0, which slows learning. Not used in hidden layers much nowadays (ReLU is preferred).

3) Tanh Activation Function:

The tanh function (short for hyperbolic tangent) is another squashing function like sigmoid, but instead of squeezing values into 0 to 1, it squeezes them into -1 to +1.

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Often used in hidden layers of neural network.

Limitation:

* Still suffers from the vanishing gradient problem when inputs are very large (positive or negative), that's why in modern deep learning, ReLU is more common in hidden layers.

4) ReLU Activation Function:

ReLU stands for Rectified Linear Unit. It's super simple.

$$f(x) = \max(0, x)$$

Use in hidden layers, works really well in CNNs, NLP, Image recognition, and many more tasks.

Limitation:

* Dying ReLU problem, sometimes neurons get stuck at 0 forever if weights update badly.

5) Leaky ReLU Activation Function:

It's just like ReLU, but with a small twist. In ReLU, whenever the input is negative, the output is 0. Leaky ReLU fix, instead of giving 0 for negative

inputs, it gives a tiny negative value like (0.01), this way the neuron will never be completely dead.

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{if } x \leq 0 \end{cases}$$

Limitation:

- * A small negative slope may bias results; the slope value needs tuning.

6) PReLU Activation Function:

PReLU (parametric Rectified Linear Unit) is an improved version of Leaky ReLU. In PReLU, the slope, which was fixed in leaky ReLU, may be learned automatically by the model during training. This makes it more flexible and adaptive.

The formula is the same as leaky ReLU.

Limitation:

- * Extra parameters, risk of overfitting, slightly more complex.

7) Swish Activation Function:

Swish is a smooth, non-linear activation function used in deep learning. It helps neural networks learn better patterns because it allows small negative values and is smoother than ReLU.

Formula: $f(x) = x \cdot \sigma(x)$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Limitation:

- It is more computationally expensive than ReLU.
- It does not always outperform ReLU on every dataset or architecture.

Usage of Activation Function:

